ROOTCAUSE SERVER

09 · APIs e integraciones

Documento: 09-apis-and-integrations.md **Versión analizada:** 0.2.0 **Commit:** 2efedb2 **Fecha del análisis:** 2026-08-26

CONTENIDO

- Características generales
- Autenticación
- Encabezados
- Catálogo de endpoints
  - GET /healthz — público
  - GET /readyz — público
  - GET /api/v1/status
  - GET /api/v1/rules
  - GET /api/v1/policy
  - GET /api/v1/assets
  - POST /api/v1/assets/register
  - GET /api/v1/assets/{id}
  - POST /api/v1/telemetry
  - GET /api/v1/incidents
  - GET /api/v1/incidents/{id} y /runbook
  - POST /api/v1/incidents/{id}/status
  - GET /api/v1/exposure
  - GET /api/v1/threats
  - GET /api/v1/topology
  - GET /api/v1/audit
  - GET /api/v1/export
  - GET /metrics
- Códigos de estado
- Límite de tasa y reintentos
- Webhooks
- Integraciones externas
  - Dependencias de proveedores externos
- Compatibilidad y versionado

> Contrato HTTP completo, autenticación, códigos de estado, formatos, exposición Prometheus, exportación de evidencia e integraciones con sistemas externos. Complementa a `docs/API.md`, que es el contrato que mantiene el equipo.

Características generales

Aspecto	Valor
Estilo	REST sobre JSON

Aspecto	Valor
Prefijo versionado	`/api/v1`
Formato	`application/json; charset=utf-8`, salvo exportación y métricas
Autenticación	Token *bearer* único y compartido
TLS	**No lo termina el servidor.** Va detrás de un proxy inverso
Compresión	gzip cuando el cliente lo negocia (`CompressionLayer`)
Tiempo de espera	30 s por petición, respuesta `408`
Cuerpo máximo	`--max-body-kib`, 1 MiB por omisión, respuesta `413`
CORS	*No configurado.* Aunque `tower-http` trae la capa, el router **no la usa**; la CSP fija `connect-src 'self'`
Versión del protocolo de agente	`1.1`; se aceptan `1.0` y `1.1`

Autenticación

Todas las rutas exigen token salvo `GET /healthz` y `GET /readyz`.

```
Authorization: Bearer <token de al menos 32 caracteres>
```

Orden de evaluación, que es una decisión de diseño registrada en el ADR 0003:

1. **Límite de tasa** por dirección (cubeta de tokens).
2. **Bloqueo** por fallos de autenticación repetidos.
3. **Comparación del token** en tiempo constante.

Comparar la credencial antes del perímetro convertiría el endpoint en un oráculo gratuito para quien esté adivinando.

Modo sin token. Con `--insecure-dev-mode` no se exige credencial, pero `ServeSettings::validate` garantiza que esa combinación solo es posible escuchando en loopback, y `GET /api/v1/status` lo declara en `hardening.authentication: false`.

Encabezados

De petición.

Cabecera	Uso
`Authorization: Bearer …`	Obligatoria salvo en rutas públicas
`Content-Type: application/json`	En `POST`
`X-Forwarded-For`	Solo se lee si el operador activó `--trust-forwarded-for`; se toma la entrada **más a la izquierda**
`X-Forwarded-Proto: https`	Si está, el servidor añade `Strict-Transport-Security`

De respuesta, en todas las rutas.

Cabecera	Valor
`content-security-policy`	`default-src 'self'; connect-src 'self'; img-src 'self' data:; style-src 'self'; script-src 'self'; font-src 'self'; base-uri 'none'; form-action 'none'; frame-ancestors 'none'; object-src 'none'`
`x-content-type-options`	`nosniff`
`x-frame-options`	`DENY`
`referrer-policy`	`no-referrer`
`permissions-policy`	`accelerometer=(), camera=(), geolocation=(), gyroscope=(), microphone=(), payment=(), usb=()`
`cross-origin-opener-policy`	`same-origin`
`cross-origin-resource-policy`	`same-origin`
`x-permitted-cross-domain-policies`	`none`
`cache-control`	`no-store` bajo `/api/`; `no-cache` en los activos de la consola
`strict-transport-security`	`max-age=31536000; includeSubDomains`, **solo** si llegó por TLS
`retry-after`	Solo en respuestas `429`

Catálogo de endpoints

`GET /healthz` — público

Solo dice que el proceso vive. No consulta la base de datos.

```
{ "status": "ok", "service": "rootcause-server", "version": "0.2.0" }
```

`GET /readyz` — público

Ejecuta `SELECT 1` contra el almacenamiento. Devuelve `500` con código `internal` si la base no responde.

```
{ "status": "ready", "service": "rootcause-server", "version": "0.2.0" }
```

`GET /api/v1/status`

Cifras de cabecera, postura de toda la flota y **cómo está configurada la instancia**.

```
{
  "service": "rootcause-server",
  "version": "0.2.0",
  "protocol_version": "1.1",
  "uptime_seconds": 3612,
  "assets_total": 12,
  "assets_online": 11,
  "open_incidents": 4,
  "critical_incidents": 1,
  "exposed_services": 7,
  "blocked_sources": 2,
  "detectors": 18,
  "posture": {
    "score": 61, "grade": "D",
    "dimensions": [ { "category": "intrusion", "score": 75, "findings": 1,
                      "summary": "1 hallazgo(s) abierto(s); el peor es de severidad alto." } ],
    "uninspected_surfaces": ["security-updates: no se encontró apt-get ni dnf para consultar
actualizaciones"],
    "computed_at": "2026-08-26T12:00:00Z"
  },
  "hardening": {
    "authentication": true, "bind_is_loopback": true,
    "rate_limit_per_minute": 600, "lockout_threshold": 10, "retention_days": 30
  }
}
```


El bloque `hardening` existe para que **la consola pueda advertir con honestidad** sobre su propia configuración.

`GET /api/v1/rules`

Catálogo publicado por el binario. 18 entradas.


```
[
  {
    "id": "exposure.service.public",
    "category": "exposure",
    "category_label": "Superficie expuesta",
    "title": "Servicio alcanzable fuera del host",
    "question": "¿Hay un servicio escuchando en una dirección que no es loopback?",
    "severity_ceiling": "critical",
    "techniques": ["T1190"]
  }
]
```

`GET /api/v1/policy`

Devuelve la `DetectionPolicy` en vigor, con la misma forma que imprime `rootcause-server policy`. Sirve para versionar la política junto al despliegue.

`GET /api/v1/assets`

Inventario. Cada entrada trae registro, primera y última señal, estado (`online` / `stale` / `offline`), últimas métricas, última superficie, rol y **su propia postura**, calculada solo con sus incidentes.

`POST /api/v1/assets/register`

Cuerpo: `AssetRegistration`. Respuesta `204 No Content`.

Idempotente: `INSERT … ON CONFLICT(agent_id) DO UPDATE`. El agente lo llama en cada ciclo antes de enviar telemetría.

Errores: `400` si el nombre de equipo, la versión, la arquitectura o las etiquetas no cumplen los límites.

`GET /api/v1/assets/{id}`

Detalle de un activo: su vista, sus incidentes y hasta **120 muestras** de historial. `404` si el identificador no existe.

`POST /api/v1/telemetry`

El endpoint central. Cuerpo `TelemetryEnvelope`.

```
{
  "protocol_version": "1.1",
  "asset": { "…": "AssetRegistration, opcional" },
  "sample": { "…": "MetricSample, obligatoria" },
  "security": { "…": "SecuritySignals, opcional" }
}
```

Respuesta `200`:

```
{
  "accepted": true,
  "incidents_touched": 2,
  "warnings": ["superficie no inspeccionada — firewall: no se encontró ufw, firewalld ni nftables en este host"]
}
```

Código	Motivo
`400`	Protocolo no admitido · muestra inválida · marca de tiempo fuera de la ventana de 24 h · identificadores que no coinciden · registro inválido · superficie por encima de los límites
`404`	El activo no está registrado
`413`	Cuerpo por encima de `--max-body-kib`
`429`	Perímetro: límite de tasa o bloqueo
`500`	Fallo de almacenamiento

Límites de la superficie: 1 024 sockets, 4 096 pares, 1 024 eventos de autenticación, 512 archivos vigilados, 64 brechas; dirección y servicio ≤ 64 caracteres, usuario ≤ 128, ruta ≤ 512, huella ≤ 128.

`warnings` **no es decorativo:** es donde el servidor devuelve al agente cada superficie que este declaró no haber podido inspeccionar, más un aviso si el sobre llegó sin superficie de seguridad.

`GET /api/v1/incidents`

Parámetros de consulta, todos opcionales:

Parámetro	Valores	Si es inválido
`status`	`open`, `acknowledged`, `resolved`	`400 bad_request`
`severity`	`info`, `low`, `medium`, `high`, `critical`	`400 bad_request`
`category`	`resource`, `exposure`, `intrusion`, `integrity`, `hygiene`, `availability`	`400 bad_request`
`asset`	UUID	`400` del extractor
`limit`	entero	—

Un valor inválido **se rechaza en lugar de ignorarse**, lo verifica la prueba `an_invalid_filter_value_is_rejected_instead_of_ignored`.

`GET /api/v1/incidents/{id}` y `/runbook`

El primero devuelve el `Incident` completo. El segundo, solo su lista de `RunbookStep`:

```
[
  { "description": "Confirma que 203.0.113.10 no es una IP legítima de tu operación …",
    "kind": "inspect", "requires_privileges": false, "reversible": true },
  { "description": "Bloquea el origen en el firewall del host (revierte con: ufw status numbered
&& ufw delete <n>).",
    "kind": "contain", "platform": "linux",
    "command": "ufw deny from 203.0.113.10 to any",
    "requires_privileges": true, "reversible": true }
]
```

El servidor nunca ejecuta un runbook. Lo escribe, marca si necesita privilegios y si es reversible, y deja la decisión a una persona.

`POST /api/v1/incidents/{id}/status`

```
{ "status": "acknowledged", "actor": "vacuna@operaciones" }
```

`actor` es opcional; por omisión `console-user`. Debe medir entre 1 y 100 caracteres. Devuelve el incidente actualizado y **escribe una entrada de auditoría**.

`GET /api/v1/exposure`

Superficie de toda la flota, ordenada por severidad descendente y después por equipo y puerto. Incluye `uninspected_assets`: los equipos sin superficie o con brecha en `listeners`.

```
{
  "generated_at": "2026-08-26T12:00:00Z",
  "public_services": 3,
  "private_services": 4,
  "entries": [
    { "hostname": "srv-db-01", "protocol": "tcp", "address": "0.0.0.0", "port": 5432,
      "scope": "public", "service": "PostgreSQL", "class": "database",
      "severity": "critical", "process": "postgres",
      "observed_at": "2026-08-26T11:59:50Z" }
  ],
  "uninspected_assets": ["srv-legacy-04"]
}
```


`GET /api/v1/threats`

Orígenes que presionan la autenticación de la flota (hasta 100) **más los contadores del propio perímetro del plano de control**:


```
{
  "generated_at": "2026-08-26T12:00:00Z",
  "total_failures": 412,
  "distinct_sources": 9,
  "sources": [
    { "source_address": "203.0.113.10", "failures": 47, "successes": 1,
      "services": ["sshd"], "usernames": ["root", "admin"],
      "assets": ["srv-db-01"], "severity": "critical",
      "first_seen": "2026-08-26T11:40:00Z", "last_seen": "2026-08-26T11:59:40Z" }
  ],
  "control_plane_defense": [
    { "reason": "auth.lockout", "count": 3, "last_seen": "2026-08-26T10:12:00Z" }
  ]
}
```


`GET /api/v1/topology`

Grafo de zonas. Nodos fijos `internet` y `rootcause-server`; nodos de zona `zone:expuesto` y `zone:interno` según haya o no puertos públicos; un nodo `asset:{uuid}` por equipo. Aristas `alcanza`, `supervisa` y `contiene`.

`GET /api/v1/audit`

Rastro inmutable, con `limit` opcional (200 por omisión).

```
[
  { "observed_at": "2026-08-26T12:00:00Z", "actor": "rootcause-engine",
    "action": "incident.auto_resolved", "target": "3f2a…",
    "detail": { "fingerprint": "8f14…:exposure.service.public:tcp/5432",
                "reason": "la condición dejó de observarse" } }
]
```

`GET /api/v1/export`

Paquete de evidencia en **NDJSON**: un documento JSON por línea, tipado por `kind`.

```
Content-Type: application/x-ndjson; charset=utf-8
Content-Disposition: attachment; filename="rootcause-evidencia.ndjson"
```

```
{"kind":"export","service":"rootcause-
server","version":"0.2.0","protocol_version":"1.1","generated_at":"…"}
{"kind":"asset","asset":{…}}
{"kind":"incident","incident":{…}}
{"kind":"audit","audit":{…}}
```

La primera línea es siempre la cabecera del paquete. Incluye hasta 1 000 entradas de auditoría.

`GET /metrics`

Exposición Prometheus, **autenticada** (a diferencia de la costumbre habitual de dejarla abierta).

```
# HELP rootcause_posture_score Puntuación de postura, de 0 a 100.
# TYPE rootcause_posture_score gauge
rootcause_posture_score 61
# HELP rootcause_incidents_by_category Incidentes abiertos por categoría.
# TYPE rootcause_incidents_by_category gauge
rootcause_incidents_by_category{category="intrusion"} 1
```

Las once métricas están listadas en 05 · Referencia técnica.

Códigos de estado

Código	Cuándo	Cuerpo
`200`	Éxito con contenido	JSON o texto
`204`	Registro de activo aceptado	Vacío
`400`	Entrada inválida	`{error, code: "bad_request"}`
`401`	Token ausente o incorrecto	`{error, code: "unauthorized"}`
`404`	Recurso inexistente	`{error, code: "not_found"}`

Código	Cuándo	Cuerpo
`408`	Petición por encima de 30 s	Generado por `TimeoutLayer`
`413`	Cuerpo demasiado grande	Generado por `RequestBodyLimitLayer`
`429`	Límite de tasa o bloqueo	`{error, code: "rate_limited"\ "locked_out", retry_after_seconds}` + Retry-After`
`500`	Fallo interno	`{error: "error interno del servidor", code: "internal"}`

Un error interno nunca revela su causa. Se registra con `tracing` y el llamador recibe un mensaje genérico; hay una prueba (`an_internal_error_never_leaks_the_cause`) que verifica que una cadena de conexión con contraseña no acaba en la respuesta.

Límite de tasa y reintentos

Mecanismo	Valor por omisión	Configurable con
Cubeta de tokens por dirección	600 peticiones/minuto	`--rate-limit-per-minute` (10 – 100 000)
Umbral de bloqueo	10 fallos de autenticación	`--lockout-threshold` (3 – 1 000)
Duración del bloqueo	300 s	`--lockout-seconds` (30 – 86 400)
Direcciones vigiladas a la vez	20 000, con desalojo de la más antigua	Constante
Olvido por inactividad	900 s	Constante

La cubeta se rellena de forma continua a `rate_limit_per_minute / 60` tokens por segundo. `Retry-After` se calcula a partir de lo que falta para el siguiente token y nunca es `0` .

Reintentos del cliente. El agente aplica reintento exponencial: duplica el intervalo tras cada fallo hasta 300 s y vuelve al normal al primer éxito. **No hay cola local**: un ciclo que no se pudo entregar se pierde, y el siguiente ciclo reporta el estado actual.

Webhooks

No hay. El sistema no emite webhooks ni notificaciones salientes. La integración con sistemas de alerta se hace hoy por Prometheus. Está declarado como planificado en `docs/ROADMAP.md` .

Integraciones externas

Integración	Dirección	Contrato	Obligatoria
Prometheus	Prometheus → servidor	`GET /metrics` con token	No
Proxy inverso con TLS	Cliente → proxy → servidor	HTTP local; `X-Forwarded-For` y `X-Forwarded-Proto`	Recomendada para uso remoto
systemd / launchd	Sistema → binario	Unidades en `packaging/`	No
Docker / Compose	Orquestador → contenedor	`compose.yml` con healthcheck sobre `/dev/tcp`	No
Herramientas del SO	Agente → binarios locales	Lista blanca de 11 programas, solo lectura, con tiempo límite	Sí, para la superficie de seguridad

Integración	Dirección	Contrato	Obligatoria
GitHub Actions	Repositorio	5 workflows	Infraestructura
GitHub Pages	`landing/`	Publicación de la página del producto	No

Dependencias de proveedores externos

En **tiempo de ejecución**: ninguna. El binario no habla con ningún servicio de terceros.

En **tiempo de compilación y CI**: crates.io, GitHub Actions y las imágenes base de Docker. Esa superficie está gestionada explícitamente:

- `Cargo.lock` versionado;
- `cargo deny` sobre licencias, orígenes desconocidos, duplicados y versiones retiradas;
- `cargo audit` con una única exención documentada y **verificada por CI** (RUSTSEC-2023-0071 sobre `rsa`, que nunca se compila porque el driver MySQL de `sqlx` no se activa);
- toda acción de GitHub pinneada a SHA de 40 caracteres, comprobado por `guard_claims.py`;
- SBOM CycloneDX generado en cada ejecución del workflow de seguridad.

Compatibilidad y versionado

Elemento	Regla
Versión del API	Prefijo `/api/v1`. Un cambio incompatible exigiría `/api/v2`
Protocolo de agente	`PROTOCOL_VERSION = "1.1"`; se aceptan `1.0` y `1.1`. Solo sube cuando el contrato deja de ser compatible; **añadir un campo opcional no cuenta**
Agente 0.1 contra servidor 0.2	Funciona: `TelemetryEnvelope.security` es opcional y el servidor lo declara como advertencia
Base de datos 0.1 contra binario 0.2	Funciona: toda columna nueva es nullable o tiene valor por omisión, e `Incident.category` tiene `serde(default)`
Compatibilidad hacia atrás del binario	*No documentado en el repositorio* para el caso de bajar de versión. Requeriría restaurar un respaldo

---

Anterior: 08 · Flujo de datos · **Siguiente:** 10 · Configuración · **Índice:** README
